

■ Terminologia

bad, função-membro de `basic_ios` 742
 badbit 726
`basic_fstream`, template de classe 724
`basic_ifstream`, template de classe 724
`basic_ios`, template de classe 855
`basic_iostream`, template de classe 723
`basic_istream`, template de classe 723
`basic_ofstream`, template de classe 724
`basic_ostream`, template de classe 723
 bibliotecas clássicas de stream 722
 bibliotecas-padrão de stream 722
 bits de estado 726
`boolalpha`, manipulador de stream 739
 buffer 724
 caractere em branco 726
 caracteres de preenchimento 731
`clear`, função-membro de `basic_ios` 743
`dec`, manipulador de stream 737
 E/S não formatada 722
 E/S seguras quanto ao tipo 721
`eof`, função-membro de `basic_ios` 878
`eofbit` 741
 estado original 740
`fail`, função-membro de `basic_ios` 741
`failbit` 726
`fill`, função-membro de `basic_ios` 736
`fixed`, manipulador de stream 738
`flags`, função-membro de `ios_base` 740
`fmtflags` 740
`fstream` 724
`gcount`, função-membro de `basic_istream` 729
`get`, função-membro de `basic_istream` 726
`getline`, função-membro de `basic_istream` 728
`good`, função-membro de `basic_ios` 742
`goodbit` 742
`hex`, manipulador de stream 737
`ifstream` 724
`ignore`, função-membro de `basic_istream` 728
`internal`, manipulador de stream 736
`iostream` 723
`istream` 723
`left`, manipulador de stream 735
 manipuladores de stream 729
 manipuladores de streams parametrizados 723
`noboolalpha`, manipulador de stream 739
`noshowbase`, manipulador de stream 737
`noshowpoint`, manipulador de stream 734
`noshowpos`, manipulador de stream 736
`noupper`, manipulador de stream 739
`oct`, manipulador de stream 737
`ofstream` 724
`ostream` 723
`peek`, função-membro de `basic_istream` 728
 precisão 730
`precision`, função-membro de `ios_base` 731
 preenchimento 731
`putback`, função-membro de `basic_istream` 728
`rdstate`, função-membro de `basic_ios` 742
`read`, função-membro de `basic_istream` 728
`right`, manipulador de stream 735
`scientific`, manipulador de stream 738
`setbase`, manipulador de stream 729
`setfill`, manipulador de stream 736
`showbase`, manipulador de stream 737
`showpoint`, manipulador de stream 734
`showpos`, manipulador de stream 736
 streams 722
`tie`, função-membro de `basic_ios` 743
`typedef` 723
 Unicode®, conjunto de caracteres 722
`uppercase`, manipulador de stream 739
`wchar_t` 722
`width`, manipulador de stream 731
`write`, função-membro de `basic_ostream` 728

■ Exercícios de autorrevisão

23.1 Preencha os espaços em cada uma das sentenças:

- a)** Entrada/saída em C++ ocorre como _____ de bytes.
- b)** Os manipuladores de stream que formatam o alinhamento são _____, _____ e _____.
- c)** A função-membro _____ pode ser usada para definir e restaurar o estado original.
- d)** A maioria dos programas em C++ que realiza E/S deve incluir o arquivo de cabeçalho _____, que contém as declarações exigidas em todas as operações de E/S de stream.
- e)** Ao usar manipuladores parametrizados, o arquivo de cabeçalho _____ deve ser incluído.

- f)** O arquivo de cabeçalho _____ contém as declarações exigidas no processamento de arquivo.
- g)** A função-membro _____ de `ostream` é usada para executar a saída não formatada.
- h)** As operações de entrada são admitidas pela classe _____.
- i)** Saídas de stream de erro-padrão são direcionadas para os objetos de stream _____ ou _____.
- j)** Operações de saída são admitidas pela classe _____.
- k)** O símbolo para o operador de inserção de stream é _____.
- l)** Os quatro objetos que correspondem aos dispositivos-padrão no sistema são _____, _____, _____ e _____.
- m)** O símbolo para o operador de extração de stream é _____.
- n)** Os manipuladores de stream _____, _____ e _____ especificam que os inteiros devem ser exibidos em formatos octal, hexadecimal e decimal, respectivamente.
- o)** O manipulador de stream _____ faz com que números positivos apareçam com um sinal de mais.

23.2 Indique quais das sentenças a seguir são *verdadeiras* e quais são *falsas*. Em caso de alternativas *falsas*, justifique sua resposta.

- a)** A função-membro de stream `flags` com um argumento `long` define a variável de estado `flags` como seu argumento e retorna seu valor anterior.
- b)** O operador de inserção de stream `<<` e o operador de extração de stream `>>` são sobrecarregados para tratar de todos os tipos de dados-padrão — incluindo strings e endereços de memória (apenas inserção de stream) — e todos os tipos de dados definidos pelo usuário.
- c)** A função-membro de stream `flags` sem argumentos reinicia o estado original do stream.
- d)** O operador de extração de stream `>>` pode ser sobrecarregado com uma função operador que utilize uma referência `istream` e uma referência a um tipo definido pelo usuário como argumentos e retorne uma referência `istream`.
- e)** O operador de inserção de stream `<<` pode ser sobrecarregado com uma função operador que utilize uma referência `istream` e uma referência a um tipo definido pelo usuário como argumentos e retorne uma referência `istream`.
- f)** É padrão que a entrada com o operador de extração de stream `>>` sempre pule os caracteres iniciais de espaço em branco no stream de entrada.
- g)** A função-membro de stream `rdstate` restaura o estado atual do stream.
- h)** O stream `cout` normalmente é conectado à tela de vídeo.
- i)** A função-membro de stream `good` retorna `true` se todas as funções-membro `bad`, `fail` e `eof` retornarem `false`.
- j)** O stream `cin` normalmente está conectado à tela de vídeo.
- k)** Se um erro irreversível ocorrer durante uma operação de stream, a função-membro `bad` retornará `true`.
- l)** A saída para `cerr` não é mantida em buffer, e a saída para `clog` é mantida em buffer.
- m)** O manipulador de stream `showpoint` força os valores de ponto flutuante a serem impressos com os seis dígitos de precisão-padrão, a menos que o valor da precisão tenha sido mudado; nesse caso, os valores de ponto flutuante imprimem com a precisão especificada.
- n)** A função-membro `put` de `ostream` imprime o número de caracteres especificado.
- o)** Os manipuladores de stream `dec`, `oct` e `hex` afetam apenas a próxima operação de saída de inteiro.
- p)** O padrão é que os endereços de memória sejam exibidos como inteiros `long`.

23.3 Para cada um dos itens a seguir, escreva uma única instrução que realize a tarefa requerida.

- a)** Imprimir a string "Digite seu nome: ".
- b)** Usar um manipulador de stream que faz com que o expoente em notação científica e as letras nos valores hexadecimais sejam impressos em letras maiúsculas.
- c)** Imprimir o endereço da variável `myString` do tipo `char *`.
- d)** Usar um manipulador de stream para garantir que os valores de ponto flutuante imprimam em notação científica.
- e)** Imprimir o endereço na variável `integerPtr` de tipo `int *`.
- f)** Usar um manipulador de stream de modo que, quando valores inteiros são impressos, a base inteira para valores octais e hexadecimais seja exibida.
- g)** Imprimir o valor apontado por `floatPtr` de tipo `float *`.
- h)** Usar a função-membro de stream para definir o caractere de preenchimento como '*' para impressão em larguras de campo maiores que os valores sendo impressos. Repita essa instrução com um manipulador de stream.

- i) Imprimir os caracteres 'O' e 'K' em uma instrução com a função `put` de `ostream`.
 - j) Obter o valor do próximo caractere a entrar sem extraí-lo do stream.
 - k) Incluir um único caractere na variável `charValue` de tipo `char`, usando a função-membro `get` de `istream` de duas maneiras diferentes.
 - l) Incluir e descartar os próximos seis caracteres no stream de entrada.
 - m) Usar a função-membro `read` de `istream` para incluir 50 caracteres no array de `char line`.
 - n) Ler 10 caracteres no array de caracteres `name`. Parar de ler os caracteres se o delimitador '.' for encontrado. Não remover o delimitador do stream de entrada. Escrever outra instrução que realize essa tarefa e remova o delimitador da entrada.
 - o) Usar a função-membro `gcount` de `istream` para determinar o número de caracteres inseridos no array de caracteres `line` pela última chamada à função-membro `read` de `istream`, e imprimir esse número de caracteres usando a função-membro `write` de `ostream`.
 - p) A saída 124, 18.376, 'Z', 1000000 e "String", separada por espaços.
 - q) Imprimir a configuração de precisão atual usando uma função-membro do objeto `cout`.
 - r) Incluir um valor inteiro na variável `int months` e um valor de ponto flutuante na variável `float percentageRate`.
 - s) Imprimir 1.92, 1.925 e 1.9258 separados por tabulações e com 3 dígitos de precisão usando um manipulador de stream.
 - t) Imprimir o inteiro 100 em octal, hexadecimal e decimal usando manipuladores de stream, separados por tabulações.
 - u) Imprimir o inteiro 100 em decimal, octal e hexadecimal separados por tabulações usando o manipulador de stream para mudar a base.
 - v) Imprimir 1234 alinhado à direita em um campo de 10 dígitos.
 - w) Ler caracteres no array de caracteres `line` até que o caractere 'z' seja encontrado, até um limite de 20 caracteres (incluindo um caractere nulo de finalização). Não extraia o caractere delimitador do stream.
 - x) Usar variáveis inteiras `x` e `y` para especificar a largura de campo e a precisão usadas para exibir o valor `double 87.4573`, e exibir o valor.
- 23.4** Identifique os erros em cada uma das instruções a seguir e explique como corrigi-los.
- a) `cout << "Valor de x <= y é: " << x <= y;`
 - b) A instrução a seguir deverá imprimir o valor inteiro de 'c'.
`cout << 'c';`
 - c) `cout << ""Uma string entre aspas"";`
- 23.5** Para cada um dos itens a seguir, mostre a saída.
- a) `cout << "12345" << endl;`
`cout.width( 5 );`
`cout.fill( '*' );`
`cout << 123 << endl << 123;`
 - b) `cout << setw( 10 ) << setfill( '$' ) << 10000;`
 - c) `cout << setw( 8 ) << setprecision( 3 ) << 1024.987654;`
 - d) `cout << showbase << oct << 99 << endl << hex << 99;`
 - e) `cout << 100000 << endl << showpos << 100000;`
 - f) `cout << setw( 10 ) << setprecision( 2 ) << scientific << 444.93738;`

Respostas dos exercícios de autorrevisão

- 23.1** a) `streams`. b) `left`, `right` e `internal`. c) `flags`. d) `<iostream>`. e) `<iomanip>`. f) `<fstream>`. g) `write`. h) `istream`. i) `cerr` ou `clog`. j) `ostream`. k) `<<`. l) `cin`, `cout`, `cerr` e `clog`. m) `>>`. n) `oct`, `hex` e `dec`. o) `showpos`.
- 23.2** a) Falso. A função-membro de stream `flags` com um argumento `fmtflags` define a variável de estado `flags` como seu argumento e retorna as configurações de estado anteriores. b) Falso. Os operadores de inserção e de extração de stream não são sobrecarregados para todos os tipos definidos pelo usuário. Você precisa fornecer as funções-membro específicas sobrecarregadas para

sobrecarregar os operadores de stream para usá-las com cada tipo definido pelo usuário que você criar. c) Falso. A função-membro de stream `flags` sem argumentos retorna as configurações atuais de formato como um tipo de dado `fmtflags`, que representa o estado original. d) Verdadeiro. e) Falso. Para sobrecarregar o operador de inserção de stream `<<`, a função operador sobrecarregada precisa usar uma referência `ostream` e uma referência a um tipo definido pelo usuário como argumentos e retornar uma referência `ostream`. f) Verdadeiro. g) Verdadeiro. h) Verdadeiro. i) Verdadeiro. j) Falso. O stream `cin` é conectado

à entrada-padrão do computador, que normalmente é o teclado. k) Verdadeiro. l) Verdadeiro. m) Verdadeiro. n) Falso. A função-membro `put` de `ostream` imprime seu argumento de caractere único. o) Falso. Os manipuladores de stream `dec`, `oct` e `hex` definem o estado original de saída para inteiros para a base especificada até que a base seja mudada novamente ou o programa termine. p) Falso. O padrão é que os endereços de memória sejam exibidos em formato hexadecimal. Para exibir endereços como inteiros `long`, o endereço deve ser convertido em um valor `long`.

23.3 a) `cout << "Digite seu nome: ";`
 b) `cout << uppercase;`
 c) `cout << static_cast< void * >( myString );`
 d) `cout << scientific;`
 e) `cout << integerPtr;`
 f) `cout << showbase;`
 g) `cout << *floatPtr;`
 h) `cout.fill( '*' );`
 `cout << setfill( '*' );`
 i) `cout.put( 'O' ).put( 'K' );`
 j) `cin.peek();`
 k) `charValue = cin.get();`
 `cin.get( charValue );`
 l) `cin.ignore( 6 );`
 m) `cin.read( line, 50 );`
 n) `cin.get( name, 10, '.' );`
 `cin.getline( name, 10, '.' );`
 o) `cout.write( line, cin.gcount() );`
 p) `cout << 124 << ' ' << 18.376 << ' ' << "Z"`
 `<< 1000000 << " String";`
 q) `cout << cout.precision();`
 r) `cin >> months >> percentageRate;`
 s) `cout << setprecision( 3 ) << 1.92 << '\t'`
 `<< 1.925 << '\t' << 1.9258;`

t) `cout << oct << 100 << '\t' << hex << 100`
 `<< '\t' << dec << 100;`
 u) `cout << 100 << '\t' << setbase( 8 ) << 100`
 `<< '\t' << setbase( 16 ) << 100;`
 v) `cout << setw( 10 ) << 1234;`
 w) `cin.get( line, 20, 'z' );`
 x) `cout << setw( x ) << setprecision( y )`
 `<< 87.4573;`

23.4 a) *Erro:* A precedência do operador `<<` é mais alta que a de `<=`, o que faz com que a instrução seja avaliada indevidamente, e também cause um erro de compilação.

Correção: Coloque parênteses em torno da expressão `x <= y`.

b) *Erro:* Em C++, os caracteres não são tratados como inteiros pequenos, como acontece em C.

Correção: Para imprimir o valor numérico para um caractere no conjunto de caracteres do computador, o caractere precisa ser convertido em um valor inteiro, como a seguir:

`cout << static_cast< int >( 'c' );`

c) *Erro:* Caracteres de aspas não podem ser impressos em uma string, a menos que seja usada uma sequência de escape.

Correção: Imprima a string da seguinte maneira:

`cout << "\"Uma string entre aspas\"";`

23.5 a) 12345
 **123
 123
 b) \$\$\$\$10000
 c) 1024.988
 d) 0143
 0x63
 e) 100000
 +100000
 f) 4.45e+002

Exercícios

23.6 Escreva uma instrução para cada um dos itens a seguir:

- a) Imprimir o inteiro 40000 alinhado à esquerda em um campo de 15 dígitos.
- b) Ler uma string em uma variável de array de caracteres `state`.
- c) Imprimir 200 com e sem sinal.

- d) Imprimir o valor decimal 100 em formato hexadecimal precedido por 0x.
- e) Ler caracteres para o array `charArray` até que o caractere 'p' seja encontrado, até um limite de 10 caracteres (incluindo o caractere nulo de finalização). Extraia o delimitador do stream de entrada e descarte-o.

- f) Imprimir 1.234 em um campo de 9 dígitos com zeros iniciais.

23.7 *Inclusão de valores decimal, octal e hexadecimal.*

Escreva um programa para testar a entrada de valores inteiros em formatos decimal, octal e hexadecimal. Imprima cada inteiro lido pelo programa nos três formatos. Teste o programa com os seguintes dados de entrada: 10, 010, 0x10.

23.8 *Impressão de valores de ponteiro como inteiros.*

Escreva um programa que imprima valores de ponteiro, usando conversões para todos os tipos de dados inteiros. Quais imprimem valores estranhos? Quais causam erros?

23.9 *Impressão com larguras de campo.*

Escreva um programa que teste os resultados de impressão do valor inteiro 12345 e do valor de ponto flutuante 1.2345 em campos de diversos tamanhos. O que acontece quando os valores são impressos em campos que contêm menos dígitos do que os valores?

23.10 *Arredondamento.*

Escreva um programa que imprima o valor 100.453627 arredondado até o mais próximo dígito, décimo, centésimo, milésimo e décimo de milésimo.

23.11

Escreva um programa que aceite uma string do teclado e determine o tamanho da string. Imprima a string em uma largura de campo que seja o dobro do tamanho da string.

23.12 *Conversão de Fahrenheit em Celsius.*

Escreva um programa que converta temperaturas Fahrenheit inteiras de 0 a 212 graus em temperaturas Celsius em ponto flutuante com 3 dígitos de precisão. Use a fórmula

```
celsius = 5.0 / 9.0 * ( fahrenheit - 32 );
```

para realizar o cálculo. A saída deverá ser impressa em duas colunas alinhadas à direita e as temperaturas Celsius devem ser precedidas por um sinal para valores positivos e negativos.

23.13

Em algumas linguagens de programação, as strings são incluídas delimitadas por aspas ou apóstrofes. Escreva um programa que leia as três strings `suzy`, `"suzy"` e `'suzy'`. As aspas e os apóstrofes são ignorados ou lidos como parte da string?

23.14 *Leitura de números de telefone usando um operador de extração de stream sobrecarregado.*

Na Figura 19.5, os operadores de extração e inserção de stream foram sobrecarregados para a entrada e saída de objetos da classe `PhoneNumber`. Reescreva o operador de extração de stream para realizar a seguinte verificação de erro na entrada. A função `operator>>` precisará ser reimplementada.

- a) Inclua o número de telefone inteiro em um array. Verifique se o número apropriado de caracteres foi inserido. Deve haver um total de 15 caracteres lidos para um número de telefone no formato (11) 5555-1212. Use a função-membro `clear` de `ios_base` para definir o `failbit` de entrada imprópria.
- b) O código de área e a central não começam nem com 0 nem com 1. Teste o primeiro dígito das partes do código de área e troque partes do número de telefone para ter certeza de que nenhum comece com 0 ou com 1. Use a função-membro `clear` de `ios_base` para definir `failbit` de entrada imprópria.
- c) O dígito do meio de um código de área costumava ser limitado a 0 ou a 1 (embora isso tenha mudado recentemente). Teste o dígito do meio para ver se é um valor 0 ou 1. Use a função-membro `clear` de `ios_base` para definir `failbit` de entrada imprópria. Se nenhuma das operações acima resultar na definição de `failbit` no caso de uma entrada imprópria, copie as três partes do número de telefone nos membros `areaCode`, `exchange` e `line` do objeto `PhoneNumber`. Se `failbit` tiver sido definido na entrada, faça o programa imprimir uma mensagem de erro e termine em vez de imprimir o número do telefone.

23.15 *Classe Point.*

Escreva um programa que realize cada uma das seguintes tarefas:

- a) Cria uma classe definida pelo usuário `Point` que contenha os dados-membro inteiros privados `xCoordinate` e `yCoordinate`, e declare funções operador sobrecarregadas de inserção e extração de stream como `friends` da classe.
- b) Defina as funções operador de inserção e extração de stream. A função operador de extração de stream deverá determinar se os dados inseridos são válidos e, caso não sejam, devem definir o `failbit` para indicar a entrada imprópria. O operador de inserção de stream não deverá ser capaz de exibir o ponto depois que houver um erro de entrada.
- c) Escreva uma função `main` que teste a entrada e a saída da classe definida pelo usuário `Point` usando os operadores de extração e inserção de stream sobrecarregados.

23.16 *Classe Complex.*

Escreva um programa que realize cada uma das tarefas:

- a) Cria uma classe definida pelo usuário `Complex` que contém os dados-membro inteiros privados `real` e `imaginary` e declara as funções operador sobrecarregadas de inserção e extração de stream como `friends` da classe.

- b) Define as funções operador de inserção e de extração de stream. A função operador de extração de stream deverá determinar se os dados inseridos são válidos e, caso não sejam, deve definir `failbit` para indicar a entrada imprópria. A entrada deverá ter a forma

$$3 + 8i$$

- c) Os valores podem ser negativos ou positivos, e é possível que um dos dois valores não seja fornecido; nesse caso, o dado-membro apropriado deve ser definido em 0. O operador de inserção de stream não deve ser capaz de exibir o ponto se tiver ocorrido um erro de entrada. Para valores imaginários negativos, um sinal de subtração deverá sem impresso no lugar de um sinal de adição.
- d) Escreve uma função `main` que testa a entrada e a saída da classe `Complex` definida pelo usuário, utilizando os operadores sobrecarregados de extração e de inserção de stream.

23.17 Impressão de uma tabela de valores ASCII. Escreva um programa que use uma estrutura `for` para imprimir uma tabela de valores ASCII para os caracteres no conjunto de caracteres ASCII de 33 a 126. O programa deverá imprimir os valores decimal, octal, hexadecimal e o valor de caractere para cada caractere. Use os manipuladores de stream `dec`, `oct` e `hex` para imprimir os valores inteiros.

23.18 Escreva um programa para mostrar que o `getline` e as funções-membro `istream` de `get` terminam a string de entrada com um caractere nulo de finalização de string. Além disso, mostre que `get` deixa o caractere delimitador no stream de entrada, enquanto `getline` extrai o caractere delimitador e o descarta. O que acontece com os caracteres não lidos no stream?